



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Semestre 1-2025
Técnicas Avanzadas de Programación



Asignación #3: Hash

Análisis de Operaciones en Hash Tables

Estudiante:
Erimar Reis CI 29.743.464

Caracas, 25 de Junio de 2025

Introducción

Las **Tablas Hash** son estructuras de datos que se basan en una observación simple, si se quisieran almacenar números en un rango $[1, N]$, se podría usar una tabla de tamaño N , donde las operaciones de inserción, búsqueda y actualización serían directas. Sin embargo, si N es muy grande, esta aproximación comprometería la eficiencia de memoria. Para superar esto, las Tablas Hash utilizan una tabla de tamaño fijo máximo M , donde $M < N$, y los índices válidos van de $[0, M - 1]$.

La idea central de una Tabla Hash es usar una función hash o de dispersión para transformar una clave en un índice válido dentro del rango de la tabla $[0, M - 1]$. Para esta investigación se utilizó el módulo $\text{clave} \% M$, este asegura que el cálculo del índice esté en el intervalo deseado. Dicho índice apunta a un slot donde se almacena el valor asociado a la clave. Las Tablas Hash son fundamentales cuando se requiere acceso a elementos en tiempo constante y se usan con frecuencia para relacionar pares clave-valor.

El principal desafío en las Tablas Hash es el manejo de colisiones, que ocurre cuando dos claves diferentes se unordered mapean al mismo índice. La eficiencia de una Tabla Hash depende en gran medida de la calidad de la función hash y de la estrategia de resolución de colisiones que se adopte.

Implementaciones del manejo de colisiones

■ Separate Chaining (Encadenamiento Separado)

En este enfoque, cada posición de la tabla hash, implementada como `vector<list<Nodo>> tabla`, contiene una lista enlazada donde se almacenan todos los elementos que, al ser unordered mapeados por la función hash, resultan en el mismo índice. Esta estructura maneja colisiones añadiendo elementos a la lista correspondiente, permitiendo que múltiples claves residan en la misma posición sin sobrescribirse. La inserción busca primero si la clave ya existe en la lista para actualizar su valor, de lo contrario, añade un nuevo nodo al final de la lista. La búsqueda recorre la lista enlazada en el índice calculado hasta encontrar la clave. La eliminación se realiza buscando la clave en la lista y, una vez encontrada, se elimina directamente de la lista utilizando el iterador, lo que acorta la cadena y no afecta a las búsquedas futuras, ya que estas simplemente recorren la lista acortada.

■ Linear Probing (Sondeo Lineal)

El sondeo lineal maneja las colisiones buscando la siguiente celda vacía o inactiva de forma secuencial en la tabla, avanzando una posición a la vez y devolviéndose al inicio de la tabla si es necesario, es decir $i = (i + 1) \% M$. La inserción calcula un índice hash inicial y luego sondea linealmente hasta encontrar una posición disponible, si la clave ya existe y está activa, se actualiza su valor. Para la eliminación, esta implementación utiliza un enfoque de "marcar" el nodo como inactivo `tabla[i]->estaActivo = false` en lugar de eliminarlo físicamente y reinsertar elementos posteriores, haciéndolo más robusto y eficiente.

■ Quadratic Hashing (Sondeo Cuadrático)

Para reducir el agrupamiento que ocurre en el sondeo lineal, el sondeo cuadrático calcula la siguiente posición de sondeo utilizando una función cuadrática: `pos = (hashBase + C1 * i + C2 * i * i) \% M`, donde i es el número de intentos y $C1$, $C2$ son constantes (en esta implementación, $C1 = 1$ y $C2 = 1$). Esto expande el rango de búsqueda de manera no lineal, ayudando a distribuir mejor las colisiones. La inserción sondea cuadráticamente hasta encontrar un espacio. Para la eliminación, se marca el nodo como inactivo `tabla[i]->estaActivo = false`. Además de marcar, esta implementación intenta reubicar los elementos subsiguientes en el clúster, similar a como se haría en el sondeo lineal si se eliminara el nodo físicamente, para intentar compactar el clúster.

■ Double Hashing

Esta técnica utiliza una segunda función hash para determinar el tamaño del "salto" en caso de colisión. La fórmula para encontrar la siguiente posición es: `pos = (hashBase + i * paso) \% M`, donde `hashBase` es el resultado de la primera función hash y `paso` es el resultado de la segunda función hash, diferente para cada

clave. Esto permite una dispersión máxima de los elementos y minimiza tanto el agrupamiento primario como el secundario. La inserción sondea utilizando este doble hash. Para la eliminación, el elemento se marca como inactivo `tabla[i]->estaActivo = false`.

Las implementaciones de Sondeo Lineal, Sondeo Cuadrático y Doble Hash aplican la estrategia de "marcar" los elementos para que la eliminación sea eficiente. La operación de búsqueda no se ve afectada negativamente por esta "marcación" ya que cuando se busca una clave, el algoritmo sigue la secuencia de sondeo (lineal, cuadrática o doble hash) y, al encontrar un nodo, verifica su estado. Si el nodo está inactivo, la búsqueda simplemente lo ignora y continúa por la tabla como si el nodo inactivo no existiera en términos de contenido, pero sí en términos de ocupar un espacio que debe ser recorrido. Esto asegura que la cadena de sondeo no se rompa y que los elementos que se insertaron después de una clave eliminada y que, por lo tanto, sondearon a través de la posición de la clave eliminada, puedan seguir siendo encontrados correctamente.

Metodología Experimental

Para evaluar el rendimiento de diferentes técnicas de manejo de colisiones en Tablas Hash se abordaron los tres experimentos planteados en el enunciado, permitiendo analizar su comportamiento en distintas operaciones clave. La metodología sigue un enfoque cuantitativo, midiendo los tiempos de ejecución bajo las mismas condiciones asegurando que los datos utilizados siempre sean los mismos en cada implementación, y así garantizar comparaciones precisas.

A fin de analizar el comportamiento de cada técnica bajo distintos escenarios, cada tabla hash está implementada en un archivo .cpp, en estos se realizan pruebas controladas que varían el tamaño de la tabla (M) y la proporción de operaciones (inserciones, búsquedas y eliminaciones) para cubrir casos dominados por un tipo de operación o combinaciones mixtas. Los experimentos utilizan una semilla para generar los datos necesarios y el archivo principal (main) de cada implementación ejecuta las pruebas según los valores de N ingresados. Para garantizar consistencia, un script en Python compila y ejecuta los tres archivos .cpp con la misma semilla, valores de M y cantidad de operaciones, este script realiza tres ejecuciones por archivo, cada una con una semilla distinta, y almacena los datos generados en archivos .txt separados. Posteriormente, un segundo script procesa estos archivos, calcula los valores promediados y genera un nuevo .txt con los resultados consolidados para cada hash.

Finalmente, un tercer script analiza los resultados de los hash con los tiempos promediados y genera gráficas comparativas para cada valor de M , así podemos evaluar el rendimiento de cada hash.

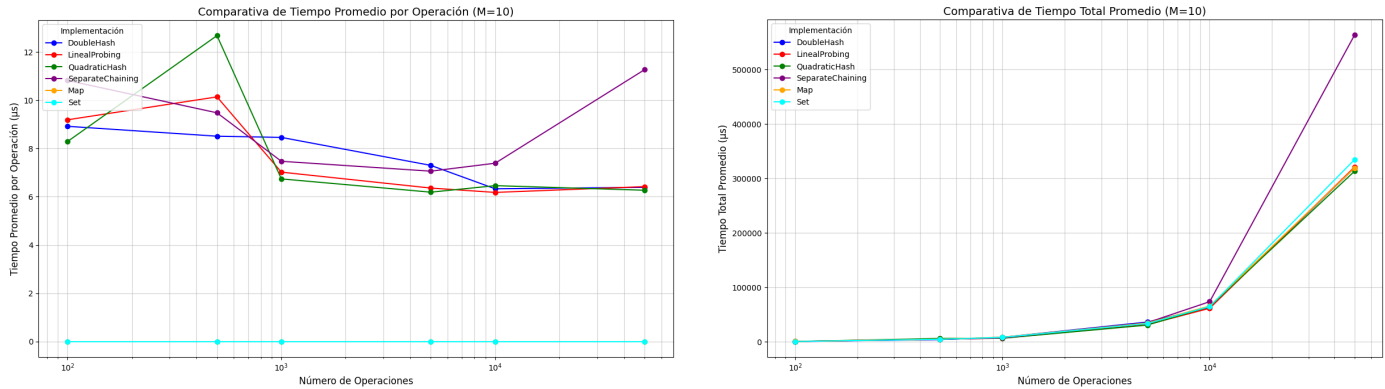
Para los experimentos se usaron las semillas [123456789, 9876, 555], los valores de M [10, 100, 1000, 29] y las cantidades de operaciones [100, 500, 1000, 5000, 10000, 50000]

Análisis de resultados

Para evaluar el rendimiento de los algoritmos implementados, se llevaron a cabo pruebas controladas en las que se midieron los tiempos de ejecución en microsegundos. A continuación, se analizan los resultados obtenidos mediante diversos gráficos que muestran el tiempo promedio de ejecución (en microsegundos) de tres operaciones fundamentales: inserción, búsqueda y eliminación. Asimismo, se presenta el tiempo total promedio para las distintas implementaciones de tablas hash (Separate Chaining, Linear Probing, Quadratic Hashing, Double Hashing, Unordered unordered map y Unordered Set) considerando múltiples configuraciones de tamaño de tabla y cantidad de operaciones. Todos los resultados provienen de cuatro escenarios experimentales clave: uso de la tabla dominado por inserciones, por búsquedas, por eliminaciones y un caso de uso mixto o promedio.

Rendimiento para M=10

Se observó el siguiente comportamiento:

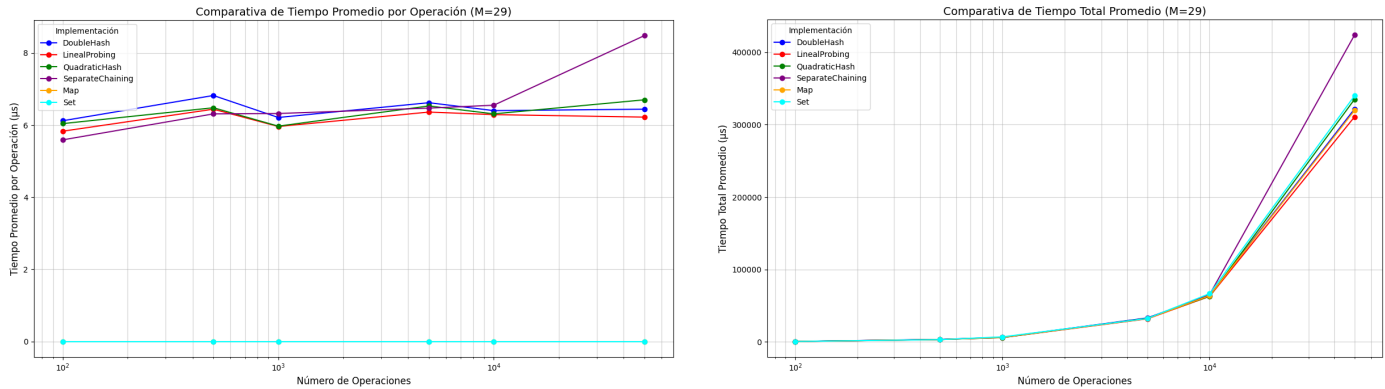


De estos resultados observamos que:

- La curva azul correspondiente a **Double Hashing** mantiene consistentemente el menor tiempo promedio por operación, incluso a medida que crece el número de operaciones. Esta eficiencia notable se debe a su estrategia de doble sondeo, que utiliza una segunda función hash para determinar los saltos. Esta técnica garantiza una dispersión más equitativa de los elementos, reduciendo significativamente el clustering y las colisiones. Aun con una tabla pequeña esta implementación logra sobresalir como la más eficiente, lo cual reafirma su superioridad en escenarios de alta saturación.
- El desempeño de **Separate Chaining** se degrada progresivamente con el aumento de operaciones. La curva morada refleja un incremento constante en el tiempo promedio, que se vuelve crítico cuando el número de operaciones alcanza valores cercanos a mil, en estos casos las listas enlazadas crecen excesivamente. Dado que las operaciones en listas largas tienen complejidad lineal, el rendimiento promedio se ve afectado. Aunque esta técnica es robusta y permite inserciones continuas, su escalabilidad se ve limitada en contextos con tablas muy pequeñas.
- La curva verde de **Quadratic Hash** muestra un comportamiento inesperado. Se observa un pico pronunciado entre las 100 y 1000 operaciones, probablemente causado por la saturación de la tabla y las dificultades del algoritmo para encontrar huecos libres debido al crecimiento de las colisiones. Este incremento podría deberse a la ausencia temporal de redimensionamiento o a una alta carga de fallos en la inserción.
- La eficiencia de Double Hashing se traduce directamente en el gráfico de tiempo total. Su curva azul permanece por debajo de las demás a lo largo de todo el experimento. Esto confirma que su rendimiento óptimo por operación se acumula de forma efectiva, manteniendo bajos los costos operativos totales incluso cuando la cantidad de operaciones crece considerablemente. En contraste, la curva morada de Separate Chaining muestra un crecimiento sostenido y elevado en el tiempo total. Dado que el tiempo por operación se incrementa gradualmente con listas enlazadas más largas, su efecto acumulativo es considerable.

Rendimiento para $M=29$

Se observó el siguiente comportamiento:

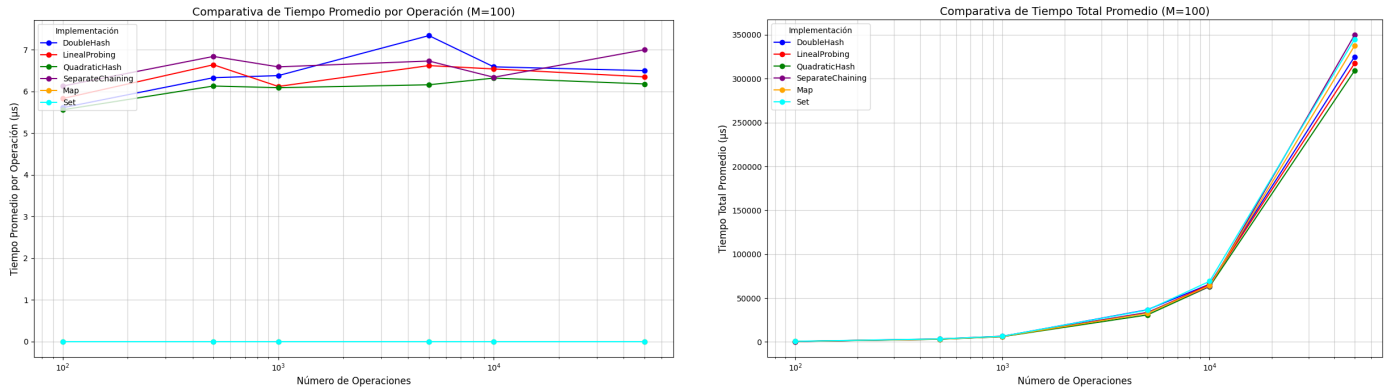


De estos resultados observamos que:

- En este experimento se aprecia un rendimiento uniforme entre la mayoría de las implementaciones. Las técnicas de sondeo Double Hashing, Linear Probing y Quadratic Hashing junto con las estructuras unordered map y unordered set presentan un tiempo promedio por operación relativamente bajo (alrededor de 6–7 microsegundos), manteniéndose estables a lo largo de un amplio rango de operaciones. Esto sugiere que con un tamaño de tabla primo como $M = 29$, la eficiencia de dispersión y acceso mejora respecto a tamaños menores.
- **Separate Chaining** aunque comienza con un rendimiento similar al del resto, la curva morada revela un pico considerable a medida que crecen las operaciones. Una vez que el número de elementos supera los 10⁴ cada lista enlazada interna contiene cientos de elementos. Esto conlleva una complejidad alta en las operaciones básicas dentro de cada lista, lo que eleva dramáticamente el tiempo promedio por operación.
- En el gráfico de tiempo total las curvas de Double Hashing, Linear Probing, Quadratic Hashing, unordered map y unordered set son cercanas entre sí y presentan un crecimiento acumulado moderado. Este comportamiento reafirma que, con una elección adecuada de M , como 29, estas estrategias logran mantener costos operativos bajos incluso cuando el número de operaciones crece. La eficiencia se sostiene gracias a una dispersión más efectiva y a una menor incidencia de colisiones. Por otro lado, la curva de Separate Chaining muestra un ascenso más pronunciado. La razón directa es el aumento progresivo del tiempo promedio por operación causado por el alargamiento de las listas internas. Así, este enfoque, aunque confiable en cuanto a capacidad de inserción, revela importantes limitaciones de escalabilidad cuando el tamaño de la tabla es insuficiente frente al volumen de operaciones.

Rendimiento para $M=100$

Se observó el siguiente comportamiento:

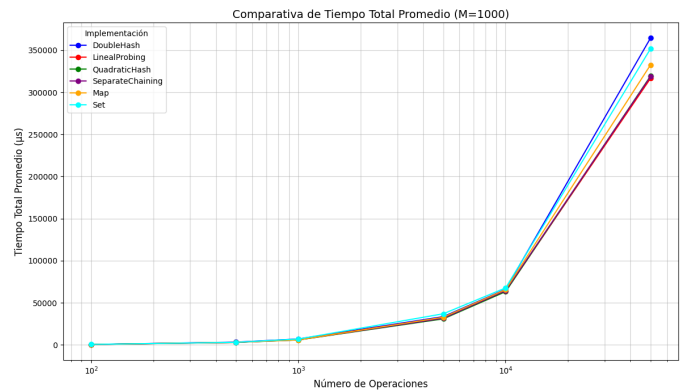
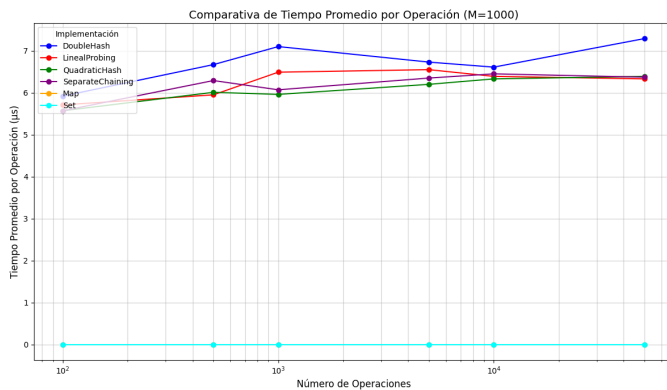


De estos resultados observamos que:

- **Quadratic Hashing** destaca como la más eficiente en casi todo el rango de operaciones. Este desempeño sugiere que, con un tamaño de tabla moderado como $M = 100$, el sondeo cuadrático logra distribuir las claves de forma efectiva, minimizando colisiones y evitando agrupamientos, aunque M no sea primo, su rendimiento práctico es sobresaliente. Esta combinación le permite sobresalir como la opción más eficiente en tiempo promedio por operación.
- A pesar de su eficiencia general, **Double Hashing** presenta un pico destacado antes de las 10^4 operaciones. Este aumento puntual podría estar asociado a una combinación de alta carga y particularidades en la función secundaria de dispersión. Es posible que, en ese tramo, la segunda función hash pierda efectividad para mantener una distribución balanceada.
- Una observación importante en este experimento es la similitud y proximidad entre las curvas. Esta convergencia sugiere que, con $M = 100$, las diferencias entre técnicas de sondeo abierto se atenúan, ya que el tamaño de la tabla reduce significativamente la incidencia de colisiones. En este contexto, todas las técnicas son razonablemente eficientes y su desempeño se aproxima, al menos en promedio.
- En tiempo total **Quadratic Hashing** se mantiene consistentemente por debajo del resto, lo que refleja su excelente comportamiento acumulado. Este resultado es coherente con su bajo tiempo promedio por operación y evidencia que el rendimiento eficiente se traduce en un costo total significativamente menor. Sin embargo, en términos generales, todas las implementaciones (incluyendo unordered map y unordered set) tienen un comportamiento similar y resultados cercano con respecto al tiempo total promedio.

Rendimiento para M=1000

Se observó el siguiente comportamiento:



De estos resultados observamos que:

- **Quadratic Hashing** muestra el mejor comportamiento de todas las implementaciones. Su rendimiento destaca por mantenerse consistentemente bajo en todo el rango de operaciones, lo cual sugiere, al igual que en los otros experimentos, una dispersión eficaz y una gestión de colisiones robusta gracias a su estrategia de sondeo no lineal. Al evitar el agrupamiento típico de otros métodos, consolidándose como una opción altamente eficiente para tablas de gran tamaño.
- Aunque teóricamente es una de las estrategias más eficientes, **Double Hashing** presenta picos inesperados en el gráfico. Estas anomalías podrían deberse a una elección inadecuada de la segunda función hash, deficiencias en la distribución de saltos o a efectos de redimensionamiento. A pesar de estos picos, su rendimiento sigue siendo competitivo.
- Una observación importante es que las curvas de todas las técnicas se mantienen bastante cercanas entre sí. Esto indica que, con un tamaño de tabla suficientemente grande, las diferencias de rendimiento promedio se reducen significativamente y todas las técnicas pueden considerarse eficientes dentro de este contexto.
- Excluyendo los picos puntuales, la mayoría de las técnicas de sondeo se comportan de forma similar, lo cual refuerza que $M = 1000$ proporciona un entorno amplio y favorable para la eficiencia de estas estructuras. Sin embargo, hay que destacar que **Quadratic Hashing** mantiene una ligera ventaja tanto en el tiempo por operación como el tiempo total.

Comparación con `Unordered map` y `Unordered unordered set`

A partir del análisis realizado sobre los distintos gráficos de rendimiento con diversos tamaños de tabla se puede concluir que las estructuras **`unordered map`** y **`unordered set`** presentan un desempeño excepcionalmente eficiente en términos de tiempo promedio por operación, superando de forma clara a todas las variantes de tablas hash implementadas. Esto se debe a su implementación interna asegura una complejidad logarítmica y una estabilidad notable incluso ante grandes volúmenes de operaciones.

Sin embargo, al observar el tiempo total acumulado, las diferencias entre **`unordered map`**, **`unordered set`** y las técnicas analizadas como **Double Hashing** y **Quadratic Hashing** tienden a reducirse considerablemente. En escenarios con tablas bien dimensionadas las implementaciones basadas en hash logran una eficiencia lo suficientemente alta como para acercarse, e incluso igualar, al rendimiento global de `unordered map` y `unordered set`. Este comportamiento sugiere que en el uso cotidiano de las estructuras, todas las técnicas analizadas ofrecen un desempeño competitivo, lo cual reafirma que la elección de una u otra estrategia puede depender tanto del patrón de uso como de restricciones específicas del sistema.

Conclusiones

De este estudio podemos concluir que **Quadratic Hashing** ofrece el mejor rendimiento global, destacando tanto en tiempo promedio por operación como en acumulado, gracias a su dispersión eficiente y su capacidad para evitar el agrupamiento sin la complejidad añadida de una segunda función hash. Sin embargo, las estructuras **`unordered map`** y **`unordered set`**, resultan ser sistemáticamente las más rápidas en cuanto a tiempo por operación debido a su estabilidad logarítmica, aunque al considerar el tiempo total acumulado, particularmente en condiciones de baja carga o tablas suficientemente grandes, las diferencias con las técnicas implementadas se reducen significativamente, lo que demuestra que todas las estrategias evaluadas pueden ser competitivas en escenarios bien configurados.

Referencias

- [Hash Functions and Types of Hash functions](#)
- [Implementation of Hash Table in C/C++ using Separate Chaining](#)
- [Why is std::unordered map implemented as a red-black tree?](#)
- [DeepSeek - AI Chat](#)